DataArt · Prod Support (AWS)

Bengaluru · Regulated banking environment · L2/L3

Search questions

0 / 56 mastered

All Topics 56 | SageMaker in Production 7 | AWS Glue 7 | Aurora DB 7 | ServiceN

YOUR EDGE FOR THIS ONE

Regulated banking is your actual background, not an analogy you have to build — Barclays and Russell Investments (via Flyingedge) both put you inside exactly this world: change control, audit requirements, incident-severity discipline. Lead with that early rather than waiting to be asked; it's a materially different starting point than a candidate who has to learn banking constraints on the job. The one gap worth being upfront about if it comes up: your certifications are Azure-side (AZ-104 / SC-300 / SC-100) against an AWS-heavy stack here — frame it the way you have before: dual-cloud fluency is the asset, not a mismatch.

Expand all | Collapse all

SAGEMAKER IN PRODUCTION

What's the operational difference between a SageMaker real-time endpoint, a serverless endpoint, and a batch transform job — and which fits a banking fraud-detection pipeline?

Real-time endpoint = a persistently running, dedicated instance (or fleet) behind an HTTPS endpoint for synchronous, low-latency single-record inference — you pay for it whether or not it's called, and you own the scaling/patching decisions via auto-scaling policies. Serverless inference removes the standing instance: AWS provisions compute on demand within a configured memory/concurrency limit, which is cost-efficient for spiky traffic but adds cold-start latency (ModelSetupTime) when scaling from zero. Batch transform is offline, high-throughput scoring of a whole dataset at once — no persistent endpoint, spins up, processes, terminates. A real-time fraud check on a live transaction needs the real-time (or provisioned-concurrency serverless) endpoint for latency guarantees; a nightly risk re-score of the full customer book is a batch transform job.

A SageMaker real-time endpoint starts throwing intermittent 5xx errors under load. Walk through your triage.

Start with Invocation5XXErrors against total Invocations — rate spike or steady background noise? Then ModelLatency and OverheadLatency, since a 5xx from a timeout looks different from one caused by the container crashing outright. Check CPUUtilization and MemoryUtilization at the instance level —

remember CPUUtilization sums across all vCPUs, so 350% on a 4-vCPU instance is near-saturation, not an anomaly. Pull the endpoint's CloudWatch Logs (/aws/sagemaker/Endpoints/<name>) for container-level stack traces — a 5xx is often the inference container itself erroring (OOM-killed, an unhandled exception in the scoring script) rather than an AWS-side fault. If errors cluster around scale-out events, suspect cold start on new instances rather than sustained load. And check whether an auto-scaling policy exists at all — fixed, undersized capacity under a traffic spike produces exactly this symptom.

How do you monitor a production SageMaker endpoint day to day, and what would you alert on?

Baseline dashboard: ModelLatency at p50/p90/p99 (not just average — tail latency is what breaches SLAs), 4XX/5XX errors as a percentage of Invocations, CPU/MemoryUtilization per instance, and InvocationsPerInstance to see whether load is balancing across the fleet. Alert on a sustained 5xx-rate crossing threshold over a few minutes (not single blips), on p99 breaching SLA, and on instance-count-vs-target-tracking mismatch. In a regulated environment, endpoint invocation logs should also flow into the same centralized logging/audit pipeline as everything else — a model's decisions need to be traceable the same way a transaction is.

How would you think about IAM for a SageMaker deployment in a banking account?

Separate roles for separate concerns, least-privilege throughout: a training/execution role scoped to specific S3 buckets/prefixes rather than account-wide access; a distinct endpoint/inference role for what the running container actually needs at inference time, which is often narrower than the training role; and human console access via SSO roles that can deploy/update endpoints without inherently having access to the training data itself. In a regulated account this usually also means VPC-only endpoints (no public path for inference traffic), KMS encryption at rest, and CloudTrail data events enabled on the buckets holding model artifacts and training data.

What deployment pattern would you use to roll out a new model version to production safely?

SageMaker supports multiple ProductionVariants behind one endpoint, each with a traffic weight — that's the native canary/blue-green mechanism. Deploy the new model as a second variant at a small weight, watch ModelLatency, error rate, and ideally a business metric (approval rate, fraud-catch rate) for that variant specifically, then shift weight gradually. A regression just means shifting weight back to the known-good variant — no redeploy needed, which matters when a fast rollback is a regulatory expectation, not a nice-to-have.

How would you talk through SageMaker cost control in a FinOps conversation?

The big levers: instance right-sizing against actual utilization; Serverless or Asynchronous Inference for spiky/low-volume workloads instead of a fixed always-on instance; multi-model endpoints when hosting many small similar models, so they share one fleet instead of each getting a dedicated one; and Savings Plans once a workload's baseline is understood. The support-engineer angle is usually surfacing the

utilization data so the platform/ML team can make the right-sizing call, rather than making the modeling decision yourself.

Why does model governance matter more in banking than in most SageMaker use cases, and what does that mean operationally?

A model scoring creditworthiness, fraud, or AML risk is making decisions with regulatory and customer-impact consequences — regulators expect the same change control, audit trail, and explainability discipline applied to it as any other production system, arguably more, because "the model changed its mind" is a harder RCA than "the code changed." Operationally: every model deployment goes through the same change-management process as any other production release, a model registry versions and approves models before they reach production, and inference logging captures enough to reconstruct why a specific decision was made if an auditor asks.

AWS GLUE

In plain terms, what does AWS Glue do in a data pipeline, and where does production support actually touch it?

Glue is serverless ETL: crawlers scan sources (S3, JDBC, etc.) and infer schema into the Glue Data Catalog, and Glue jobs — usually Spark, sometimes Python shell or Ray — transform and move data between stages, e.g. raw transaction logs in S3 into a cleaned, partitioned table Athena or Redshift Spectrum can query. Production support rarely touches the ETL logic itself; you're watching job-run health, DPU/cost efficiency, catalog schema drift, and downstream failures when a job silently produces bad or incomplete data.

A Glue job that runs nightly starts failing. What's your triage sequence?

Pull the specific run's error from the Glue console/CloudWatch Logs first — Glue surfaces a clear failure reason (script error, OOM, timeout, permissions) rather than a generic code. Access-denied points at the job's IAM role against the source/target bucket or JDBC connection, often after a bucket-policy or security-group change. An OOM/executor-lost error points at worker type/DPU count versus data volume — a job that worked for months can start failing simply because upstream volume outgrew what G.1X workers can hold in memory. If the job "succeeds" but downstream data looks wrong, suspect schema drift — the crawler picked up a new or renamed column and the job's assumptions no longer match the catalog.

What's the difference between G.1X, G.2X, and the larger G.4X/G.8X worker types, and when do you size up?

Each is measured in DPUs — one DPU is 4 vCPU and 16 GB memory. G.1X (1 DPU) and G.2X (2 DPU) are standard for most transforms/joins/queries; G.4X (4 DPU) and G.8X (8 DPU) exist for genuinely heavy jobs — large shuffles, skewed aggregations, or a Spark driver that needs more headroom because

the query plan itself is large, not just the data. There's also a memory-optimized R-type worker (roughly double the memory per DPU) for memory-bound Spark jobs that aren't necessarily CPU-heavy. If a job repeatedly OOMs or its runtime is dominated by GC/spill-to-disk, that's a worker-sizing conversation, not necessarily a code bug.

What's the FLEX execution class, and would you use it for a production job?

STANDARD is for time-sensitive jobs needing fast, dedicated startup — the default for anything with an SLA. FLEX trades startup time and interruptibility for lower cost by using spare compute capacity, similar in spirit to EC2 Spot — a good fit for non-urgent, delay-tolerant batch jobs, a poor fit for anything feeding a time-sensitive downstream process, since a FLEX job can take meaningfully longer to start.

What are Glue job bookmarks, and what's the classic gotcha?

Bookmarks track what data a job has already processed so a re-run only picks up new or changed data instead of reprocessing everything — essential for incremental pipelines. The classic gotcha: bookmark state getting out of sync with reality. Manually backfill or replay source data without resetting the bookmark, and the job silently skips it because it thinks that data is already processed; reset a bookmark without understanding what's already landed downstream, and you duplicate data instead. Any "why is yesterday's data missing" investigation on a bookmarked job should check bookmark state before assuming a code bug.

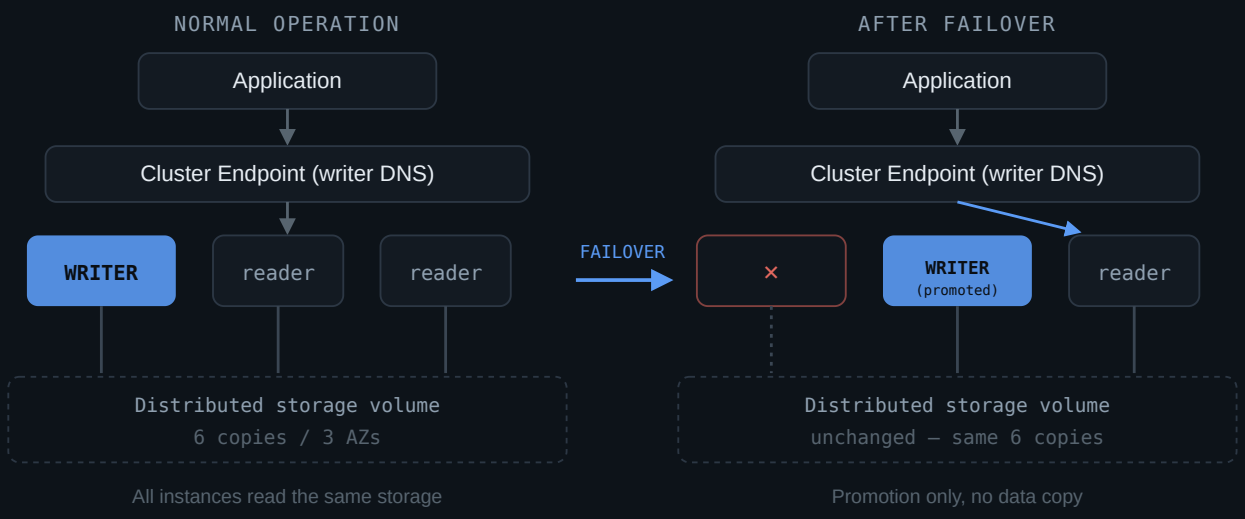
How does the Glue Data Catalog relate to Athena and Redshift Spectrum, and what happens if a crawler mis-infers a schema?

The Data Catalog is the shared metadata store — Athena and Redshift Spectrum both query S3 data directly using the Catalog's table definitions, so a crawler is effectively populating the metadata layer other services trust. A mis-inferred type (a usually-numeric column with one stray text value, so the crawler infers string instead of int, or the reverse and later rows get silently nulled) means every downstream query inherits the wrong schema until someone notices results look off or a query errors outright. Source schema changes are a "check the catalog" moment, not just a "did the job succeed" moment.

A Glue-based pipeline has gotten expensive. How do you approach the tuning?

Start with actual DPU-hours consumed (Glue bills by DPU-hour, and Cost Explorer will show which jobs dominate) rather than guessing. Common wins: right-sizing worker type/count down if utilization is low, enabling bookmarks so unchanged data isn't reprocessed every run, partitioning source/target data so jobs scan only what they need instead of full-table scans, and FLEX for non-urgent jobs. If runtime is dominated by a small number of straggler tasks, that's usually data skew — one partition much larger than the rest — which more DPUs won't fix as well as fixing the partitioning scheme would.

AURORA DB



Aurora failover: a reader is promoted in place — the storage layer never moves. Typically under 30 seconds.

What actually makes Aurora different from "RDS running MySQL/Postgres," architecturally?

Standard RDS Multi-AZ replicates the whole database engine to a standby via synchronous block-level replication. Aurora separates compute from storage: the storage layer is a distributed, self-healing volume that replicates six copies of your data across three Availability Zones automatically, and any Aurora instance — writer or reader — reads from that same shared storage layer rather than holding its own copy. That's why replicas can be added or removed without a full data copy, why failover is fast (a replica is already reading off the storage the primary was writing to — there's no data to catch up on), and why storage auto-grows in the background with no resize operation.

Walk through what actually happens during an Aurora failover, and what an application needs to handle correctly.

On a writer failure, Aurora promotes an existing Aurora Replica to writer — no data copy needed, since that replica was already reading off the same shared storage, so promotion is typically well under 30 seconds. What changes for the application is the cluster (writer) endpoint's DNS, which starts resolving to the new instance. The failure mode to watch for: connection pools or drivers that cache a resolved IP and ignore the deliberately short DNS TTL keep sending writes to the old, now-demoted instance until forced to reconnect — exactly the problem RDS Proxy or a failover-aware driver solves, by holding the logical connection open and re-pointing it under the hood instead of relying on client-side DNS re-resolution.

Cluster endpoint, reader endpoint, custom endpoint, instance endpoint — what's each one for?

Cluster (writer) endpoint always points at the current primary — that's what your application uses for writes, and it moves automatically on failover. Reader endpoint load-balances across all available Aurora Replicas, good for generic read traffic. Instance endpoints target one specific instance by name — mainly for admin/diagnostic work where you deliberately want a particular node. Custom endpoints let you define your own subset of instances (e.g., larger replicas reserved for a reporting workload) so specific traffic can be routed away from the general reader pool.

What's Aurora Backtrack, and what are its real limits?

Backtrack rewinds an Aurora MySQL cluster to an earlier point in time — seconds, not a restore-from-snapshot — by replaying a retained log of change records within a configured window, up to 72 hours. Genuinely useful for "someone ran an UPDATE without a WHERE clause." The limits worth knowing cold: MySQL only, no PostgreSQL support; incompatible with binary logging unless you force it, and forcing can break anything depending on binlog; and it cannot be used at all on a cluster that's part of an Aurora Global Database — so a globally-replicated production cluster doesn't get this particular safety net.

How does Aurora Global Database work, and what RPO/RTO does it actually give you for DR?

A primary cluster in one region plus up to five read-only secondary clusters in other regions, kept in sync via dedicated storage-layer replication rather than the database engine doing logical replication — which is why it doesn't add load to the primary's compute. AWS documents typical cross-region replication lag under one second and promotion of a secondary to full read/write in under a minute during a regional event — in practical DR terms, roughly an RPO of ~1 second and an RTO of under a minute, which is exactly why it comes up in banking DR/BCP conversations where regulators want a real number, not "we have backups."

A downstream team says Aurora read-replica data looks "stale" versus the writer. How do you investigate?

In-cluster Aurora replication is normally sub-10ms since replicas share the primary's storage, so meaningful lag is a real signal, not something to shrug off. Check AuroraReplicaLag for that specific instance — a lagging replica is often one under heavy read load or blocked by a long-running query holding up change replay. If it's a Global Database secondary in another region rather than an in-cluster replica, that's a different, normally-asynchronous-but-still-sub-second metric — AuroraGlobalDBReplicationLag — and some lag there is expected under a large write burst; the metric tells you whether it's within normal bounds or actually falling behind.

When would you reach for Aurora Serverless v2 in a production support context, and what's the trade-off?

Good fit for genuinely variable or unpredictable demand — dev/test, sharp daily/weekly peaks, or a new service where steady-state capacity isn't known yet — since it scales capacity automatically instead of

provisioning for peak. The trade-off worth naming: it's still capacity-based scaling with some reaction time, not instant, so it's not a substitute for correctly-sized provisioned capacity on a latency-sensitive, consistently-loaded core banking workload. Bursty/uncertain workloads are the fit; predictable, mission-critical ones stay on provisioned instances.

SERVICENOW & CHANGE MGMT

In ServiceNow ITSM terms, how do Incident, Problem, and Change records relate to each other?

Incident is "service is degraded or down right now, restore it" — the focus is speed, not root cause. Problem is opened, often linked from one or more incidents, when you need to actually find and fix the underlying cause, which may take longer than any single incident's resolution. Change is what you open to make the actual fix to a production system. A real RCA flow often looks like: Incident (restore service, maybe via workaround) → Problem (root-cause investigation, linked to the incident) → Change (the permanent fix, through approval) → Problem closed once the change is verified in production.

Standard, Normal, and Emergency change — what's the practical difference, and which fits a routine EKS deployment vs. a hotfix during an active P1?

Standard changes are pre-approved, low-risk, repeatable procedures that don't need a fresh CAB review each time — a routine, well-tested deployment on an established runbook can often be pre-classified this way once it's succeeded enough times. Normal changes are the default for anything with real risk or novelty, going through technical/business/CAB approval with a defined lead time. Emergency changes exist for exactly the P1-hotfix case: the fix is implemented immediately to restore service, and the change record is raised and formally reviewed after the fact, often by a smaller Emergency CAB, rather than blocking the fix on a full review cycle.

A regulator asks "show me that no unauthorized change touched this system." What in your process actually answers that?

The Change record itself is the primary evidence: who requested it, who approved it with the chain preserved, the implementation and rollback plan attached, and a timestamped link to what was actually deployed — commit hash, pipeline run. Segregation of duties matters here too — showing the approver wasn't the same person who executed it. Underneath that, CloudTrail and the deployment pipeline's own audit log corroborate that what was approved is what actually ran; the ServiceNow record and the technical trail need to agree, which is exactly the gap an auditor probes for.

What's the CMDB, and why would a production support engineer actually care about it?

The Configuration Management Database is ServiceNow's model of every system or service and how they depend on each other — Configuration Items and their relationships. Practically, it's what turns "the payments API is down" into "here's the exact blast radius" during an incident — which

upstream/downstream services, which team owns each, what's the business-criticality tier — instead of that knowledge living only in someone's head. A stale CMDB is a real production-support risk: you can under- or over-estimate impact during triage if the dependency map doesn't reflect what's actually running.

How would you push back constructively if the change process feels like it's slowing down a genuinely urgent fix?

Use the tool built for exactly this — an Emergency Change — rather than skipping process; the point of the emergency path is that urgency and governance aren't actually in conflict when a fast lane already exists. If it's not urgent enough for emergency but the normal lead time genuinely doesn't fit, that's a case for an Expedited/Normal change with the risk and business impact clearly stated, so the CAB makes an informed speed-versus-risk call rather than you making it unilaterally. The framing that lands well in a regulated environment: "I want to move fast in a way that's still defensible if someone asks me to justify it afterward," not "the process is in my way."

DR & BCP FOR BANKING

RTO vs RPO — define both, and give a banking-flavored example of why they're not the same conversation.

RPO (Recovery Point Objective) is how much data you can afford to lose, measured backward from the failure. RTO (Recovery Time Objective) is how long the service can be down, measured forward to restoration. A core-banking ledger might tolerate a slightly longer RTO — a few minutes of downtime is bad but survivable — while having an extremely tight RPO, since losing even seconds of committed transactions isn't acceptable. That's exactly why Aurora Global Database's ~1-second RPO / <1-minute RTO profile gets cited specifically for this kind of workload rather than a cheaper backup-and-restore approach with an RPO measured in hours.

Name the standard DR strategy tiers and roughly where each sits on cost vs. RTO/RPO.

Backup & Restore — cheapest, slowest, RTO/RPO in hours, restoring from backups into fresh infrastructure. Pilot Light — minimal core infrastructure always running in the DR region (e.g. a database kept in sync), rest scaled up on failover, RTO in tens of minutes. Warm Standby — a scaled-down but fully functional copy of the full stack always running in DR, scaled up on failover, RTO in minutes. Multi-Site Active-Active — full production capacity live in two-plus regions simultaneously, RTO near-zero, highest cost. The right tier is a business decision per system based on what an hour of downtime actually costs that service, not one answer for the whole bank.

What does a DR test actually need to prove in a regulated environment, beyond "we could restore the database"?

That the business function works end to end from the DR environment — application connectivity, DNS/traffic failover, downstream integrations like payment rails and third-party APIs, authentication/IAM, and that staff can actually execute the runbook under time pressure. A technically-successful database restore that nobody can route live traffic to, or that skips steps a human would forget under pressure, hasn't proven business continuity — and regulators specifically ask about test scope and frequency because "we tested the database" is a common, and inadequate, answer.

How is BCP different from DR, and why does the distinction matter when you're the one writing runbooks?

DR is the technical recovery of systems and data. BCP is broader — people, process, facilities, communications: where staff work if the primary office is unavailable, who has authority to declare a disaster and communicate to customers or regulators, what the manual fallback is if a system stays down longer than expected. A production-support runbook is naturally DR-flavored, but a good one nods to the BCP layer too — who to notify, what the customer-facing status message is, not just "run this failover command."

Does your incident response change once you know a P1 is on a system in regulatory scope vs. a purely internal tool?

The technical triage steps don't really change, but two things get added: a lower threshold for formally declaring the incident and notifying the right stakeholders — Risk/Compliance often need near-real-time visibility, not after-the-fact — and much stricter discipline about acting only through approved channels, an Emergency Change rather than an ad-hoc manual fix outside any recorded process, even under time pressure. In a regulated system, how you fixed it can matter almost as much as that you fixed it.

How would you talk through a real recovery-time improvement you've delivered, if asked in this context?

Bring your own numbers here — you've got legitimate, quantified DR-adjacent material: the recovery-time improvement tied to your chaos-engineering/resilience work, and the detection-time improvement from the Terawe/Synapse work. Lead with the failure mode you found, the specific change you made, and the measured before/after; a regulated-banking interviewer will care more about how you validated the improvement was real than about the headline number itself.

AUDIT LOGGING & COMPLIANCE

CloudTrail vs. CloudWatch Logs — what's each actually for, and why does a banking audit specifically want CloudTrail?

CloudWatch Logs is generic log storage — application logs, Lambda output, whatever you send it. CloudTrail specifically records API activity across the account: who (which IAM identity) called which API, when, from where, and what changed — the record of what happened to the infrastructure, not

application behavior. Auditors want CloudTrail specifically because it's the tamper-evident record that a given change was made by an authorized identity through an authorized path, independent of whatever the application or the ServiceNow change record claims happened.

How do you make sure CloudTrail logs themselves can't be quietly altered or deleted after the fact?

Enable log file validation — CloudTrail generates a digitally-signed digest per period, so tampering with a delivered file is detectable. Deliver logs to a dedicated, tightly locked-down S3 bucket, ideally in a separate logging/audit account so even admins in the workload account can't delete them, with a bucket policy denying delete, MFA-delete enabled, and versioning/Object Lock for good measure. The design principle interviewers are listening for: the people who can change production shouldn't be the same people who can erase the record that they changed it.

Where does GDPR-style data-privacy regulation create tension with the audit-logging requirement above, and how would you resolve it?

Right-to-erasure expects personal data to genuinely be deletable on request, while audit/compliance logging wants an immutable, complete record — those pull in opposite directions when a log line contains personal data. The usual resolution is architectural, not a policy compromise: keep security/audit logs about actions (who did what to which resource ID) largely free of personal data in the first place, and where personal data must be logged — which customer a transaction touched — separate that into a system designed for lawful retention and erasure rather than mixing it into the immutable security trail. Naming the tension explicitly is a better answer than pretending it isn't there.

What does "segregation of duties" mean concretely in a CI/CD pipeline for a regulated system?

The person who writes the code shouldn't be the same person who approves the change to production, and the approver shouldn't be the only one able to execute the deployment unreviewed. In practice: mandatory PR review from someone other than the author, a separate change-approval step in ServiceNow distinct from the merge, and pipeline permissions structured so one engineer can't merge, approve, and deploy to prod solo — even if they technically have access to each step in isolation.

How would you design logging so an investigation can reconstruct "what did the customer see," without over-collecting sensitive data?

Log decisions and outcomes by default — request ID, timestamp, service, response code/latency, which rule or model version decided — rather than raw payload content, and correlate via a request/trace ID that lets you stitch the full path together across services without every log line carrying the sensitive fields itself. If payload-level detail is genuinely needed for a specific investigation, that's a scoped, access-controlled, time-boxed lookup against a separate system with its own retention and access policy — not something sitting in the general-purpose index everyone on the team can query.

L2/L3 & REGULATED SCENARIOS

How do you draw the line between L2 and L3 in production support, in your own words?

L2 is triage, known-issue remediation, and escalation judgment — working from runbooks and dashboards, pattern-matching against past incidents, with the job being to restore service fast and know when something is genuinely novel. L3 is where you go when the runbook doesn't cover it — deep systems knowledge, reading source/config/infra to understand why, often the person who ends up writing the next runbook entry or the actual fix. The boundary is fuzzy and experience-dependent in practice, but the honest split is: L2 restores service, L3 explains why it happened and closes the gap.

Change freeze is in effect ahead of a regulatory audit, and a genuinely important — but not P1 — fix is ready to deploy. What do you actually do?

Respect the freeze — it exists precisely so the environment is stable and well-understood during the audit window, and "important but not urgent" is the textbook case a freeze is meant to hold. Document the fix, get it queued and pre-approved so it deploys the moment the freeze lifts, and if there's real business cost to waiting, raise that explicitly with whoever owns the freeze exception process rather than making the call solo. If it turns genuinely urgent — actively causing customer or compliance impact — that's when it graduates to an Emergency Change conversation, freeze or not.

How do you balance MTTR pressure against change-control requirements when they seem to pull in opposite directions?

They're usually not actually in tension if you use the tools built for this — an Emergency Change gets you speed without skipping governance. The trap is treating "the process is slow" as license to work entirely outside it under pressure: a fast, undocumented fix that later can't be explained to an auditor creates a worse long-term problem than a few extra minutes of documentation would have. MTTR is about the customer-facing outage; audit-readiness is about being able to prove, afterward, that it was handled responsibly — good process supports both if you use the right change type.

A frustrated stakeholder wants a root cause within the hour, but the RCA genuinely needs more time. How do you handle that conversation?

Separate "when is service restored" from "when do we know root cause" explicitly and early — most of the pressure comes from those two being conflated. Give an honest status: what's confirmed, what's still hypothesis, and a specific time for the next update, rather than a guess dressed up as a conclusion just to end the conversation. A wrong RCA delivered fast is worse than a right one delivered on a stated timeline, especially where the RCA itself may become a document of record.

What does your day-to-day actually look like beyond "putting out fires"?

A meaningful share is proactive, not reactive: reviewing dashboards and alert trends for early signals before they become incidents, closing action items from the last RCA so the same failure doesn't recur, tuning alert thresholds so the team actually trusts what pages them, and chipping away at automation for the manual steps that keep showing up in runbooks. This is a good spot for your real alert-noise-reduction and manual-deployment-reduction numbers if the question goes deeper.

PYTHON & AUTOMATION

What kinds of things would you actually automate with Python in an L2/L3 support role, concretely?

Log parsing and pattern extraction — scanning CloudWatch/Splunk output for a known error signature and summarizing frequency/scope instead of eyeballing it. Routine diagnostics turned into scripts, so triage starts with a "collect the standard evidence set" run the moment an alert fires, not with a person gathering it by hand. Boto3-based health checks and remediation for well-understood failure patterns. And integration glue — e.g. a script that opens or updates a ServiceNow incident automatically from a CloudWatch alarm via the ServiceNow REST API, so the audit trail starts immediately instead of after someone remembers to log it.

How would you approach writing an auto-remediation script, given the change-control context we just covered?

The remediation logic still lives inside an approved change — auto-remediation isn't an exception to change control, it's a pre-approved Standard Change that happens to execute automatically instead of by hand. Scope it tightly to a known, well-tested failure signature rather than taking broad corrective action on a fuzzy match, log every action with enough detail to reconstruct what it did and why, and build in a circuit breaker — if it fires more than a set number of times in a window, stop and page a human instead of looping, since a runaway auto-remediation script becomes its own incident.

Give a concrete example of how you'd parse CloudWatch Logs for a specific error pattern.

Query the log group with Boto3's filter_log_events for a time window using a filterPattern matching the known signature — say a specific exception name or HTTP status — then aggregate matches by a dimension that matters, like which instance or which endpoint, rather than just counting them: a raw count says it's happening, a breakdown says where to look next. Wrapping that in a scheduled Lambda rather than a cron job on an EC2 box is usually the production-friendly version, since it removes the "what if the box running the script is the thing that's down" problem.

How do you think about testing and reviewing automation scripts that touch production, versus application code?

Same discipline, arguably a higher bar, because an operational script often runs with broad permissions and no human in the loop at execution time. That means code review like any other production change, a dry-run/read-only mode you can point at production safely before enabling the mutating path, and — the one people skip — a documented rollback for the script itself, not just for what it changes, in case the automation turns out to be the thing that's wrong.

KAFKA & REST APIS

Consumer lag is climbing on a Kafka topic feeding a downstream pipeline. What's your triage?

First establish whether it's a throughput problem — consumers processing slower than producers publish, check per-message processing time and consumer CPU — or a partition-balance problem, lag concentrated on a few partitions rather than spread evenly, often too few consumers for the partition count or a hot key skewing traffic to one partition. Check consumer-group health for rebalancing churn, since a group stuck in repeated rebalances processes almost nothing while it does so. The fix is usually one of: add consumers up to the partition count, speed up per-message processing, or address the data skew — more partitions alone doesn't help if the same key keeps landing on the same one.

What's the operational risk of a REST API dependency called synchronously, and how do you defend against it?

A slow or failing downstream without protection turns into a cascading failure — your own threads/connections pile up waiting on it, and you go down too even though your own code is fine. Standard defenses: a sane timeout, not the default which is often far too long; a circuit breaker so repeated failures stop generating new attempts and fail fast instead; and, where the call doesn't need to be synchronous, moving it off the request path entirely into a queue.

How would you health-check a REST API dependency meaningfully, beyond "does it return 200"?

A 200 on a trivial health endpoint tells you the process is up, not that it can do its job — a deeper readiness check that exercises a real, cheap dependency path (can I reach my own database, not just am I running) catches more real failure modes. The trade-off worth naming: a too-deep health check can itself become a source of false alarms or extra load, so it's a balance, not "always check everything."

Where does Kafka typically sit in a banking data pipeline, and why event streaming instead of batch there?

Anywhere the business needs a near-real-time, ordered, replayable record of events — transaction streams feeding fraud detection, account-activity feeds for downstream systems, an audit-event bus — because Kafka gives both low-latency delivery and a durable, replayable log in one system, useful for

reprocessing if a downstream consumer had a bug, which a pure batch pipeline can't offer without significant extra machinery.

LEADERSHIP & BEHAVIORAL

"Tell me about your experience supporting systems in a regulated financial environment."

This is one where you have real, direct background to lead with — Barclays and Russell Investments, via Flyingedge, are both financial-services environments, so you're describing what you actually did rather than building an analogy from an unrelated industry. Bring your own: the specific systems, the compliance/change-control process you operated inside, and a concrete moment where the regulated context changed how you handled something.

"Tell me about a time you disagreed with Risk or Compliance about a proposed change."

A strong answer shows you took the constraint seriously rather than trying to route around it — explaining the technical trade-off in terms Risk/Compliance actually cares about (customer impact, data exposure, auditability) rather than pure engineering elegance, and finding a version of the change that satisfied both. Bring your own example: what you wanted, what the concern was, how you adjusted the approach, what shipped.

"How do you decide what to automate first when you're new to a production-support rotation?"

Start from what's actually paging people and how often — the highest-frequency, lowest-judgment manual tasks are the best automation ROI, not the most technically interesting ones. A framework worth stating out loud: frequency × manual effort × error-proneness, roughly in that priority order, validated with whoever's actually on call rather than assumed from the outside.

"How do you track whether someone on your team is underperforming, and on what basis?"

Concrete, observable signals over time rather than a single bad week: incident-handling quality — not just speed, did the mitigation actually hold, was the RCA solid — whether on-call load and escalation pattern looks different from peers on the same rotation, and whether documented 1:1 feedback is actually landing and changing behavior over a few cycles. Worth stating the framework: separate a skill gap, which is coachable and needs training or pairing, from an effort or fit gap, which needs a harder, more direct conversation — conflating the two is the common mistake.

"Walk me through your incident-management philosophy in a sentence or two."

Something like: restore service first, understand root cause second, and treat every RCA as a commitment to close the loop, not a formality — because an incident that recurs three months later for the same reason is a bigger trust problem than the original incident was. Bring your own real incident as the illustration once asked to go deeper.

"What's the hardest part of production support specifically, as opposed to feature development?"

Being accountable for systems you often didn't design and can't always change quickly, under time pressure, with imperfect information — which rewards a different skill than building something new: fast, disciplined hypothesis-narrowing, and the judgment to know when you're confident enough to act versus when to escalate or buy time. Bring your own concrete example if pushed for specifics.

Scope note: this handbook concentrates on what's *distinctive* to this specific posting — SageMaker as ops-supported infrastructure, Glue, Aurora internals, ServiceNow/change-type mechanics, DR-BCP framed for banking, and audit-logging tension points. General AWS, EKS/Kubernetes, Terraform, Docker, and core ITIL are deliberately kept light here on the assumption they're already covered in depth elsewhere in your prep material — say the word if that assumption is wrong for this one and it should stand alone instead.